

INTERFACING TEMPERATURE SENSOR WITH nodemcu ESP 12E MODULE

Ex.No: 1

Date: 19/12/2025

AIM:

To measure the temperature using nodemcu board and temperature sensor.

PROCEDURE:

- There are three pins in the LM35 sensor, out of which two are for power and one is for output data transmission. We have to connect all the pins to nodemcu.
- Connect the Vcc pin of the LM35 to the 3.3V pin on the nodemcu.
- Connect the GND pin of the LM35 to any GND pin of the nodemcu.
- Connect the OUT pin of the LM35 to any analog pin on the nodemcu, for example A0.
- At last, we can check the interfacing of LM35 temperature sensor with Arduino.

COMPONENTS REQUIRED:

- Nodemcu ESP 12 Module
- LM35 Temperature Sensor
- Bread Board
- Jumper Wires
- Micro USB Cable

CONNECTION TABLE:

Nodemcu	LM35 Temperature Sensor
VV, Vin (+3V)	(V) Vcc (positive +)
G, GND (Ground)	(G) GND (Ground -)
Analog Pin (A0)	Vout pin

SOURCE CODE:

```
float temp;  
  
int tempPin = 0; void setup() {  
  
Serial.begin(9600);
```

```
}  
  
void loop() {  
  int sensorvalue = analogRead(temppin); float voltage = sensorvalue * (5.0 / 1023.0); temp = voltage /  
  0.1; Serial.print("TEMPERATURE = "); Serial.print(temp);  
  Serial.print("oC"); Serial.println(); delay(1000);  
}
```

OUTPUT:

```
TEMPERATURE = 32.55oC  
TEMPERATURE = 32.45oC  
TEMPERATURE = 32.31oC  
TEMPERATURE = 31.87oC  
TEMPERATURE = 31.43oC  
TEMPERATURE = 32.75oC  
TEMPERATURE = 32.80oC  
TEMPERATURE = 32.84oC
```

RESULT:

Thus, through the help of LM35 temperature sensor interfacing with nodemcu, the temperature is calculated and recorded.

INTERFACING SOIL MOISTURE SENSOR WITH nodemcu ESP 12E

MODULE

Ex.No: 2

Date: 9/1/2026

AIM:

To measure the moisture percentage using nodemcu board and soil moisture sensor.

PROCEDURE:

- There are three pins in the DHT11 sensor, out of which two are for power and one is for output data transmission.
- We have to connect all the pins to nodemcu.
- Connect the Vcc from the Amplifier to the 3.3V pin on the nodemcu.
- Connect the GND pin to the GND pin on the nodemcu (VIN pin).
- Connect nodemcu to PC via USB cable.
- Connect Analog pin to the nodemcu (A0) to (OUT pin).
- After completing the wiring connections, insert sensor into the soil.

COMPONENTS REQUIRED:

- nodemcu (ESP 12 Module)
- Moisture Sensor
- Breadboard
- Jumper Wires
- Micro USB Cable

CONNECTION TABLE:

Nodemcu	Soil Moisture Sensor
VV, Vin (+3V)	(V) Vcc (positive +) Amplifier
G, GND (Ground)	(G) GND (Ground -)
Analog Pin (A0)	Vout pin

SOURCE CODE:

```
const int sensor_pin = A0; void setup() {  
  Serial.begin(9600);  
}  
void loop() {  
  float moisture_percentage; int sensor_analog;  
  sensor_analog = analogRead(sensor_pin);  
  moisture_percentage = (100 - ((sensor_analog / 1023.00) * 100));  
  Serial.print("Moisture Percentage"); Serial.print(moisture_percentage);  
  Serial.print("% \n \n"); delay(1000);  
  
}
```

OUTPUT:

```
Moisture Percentage0.00%  
Moisture Percentage31.48%  
Moisture Percentage42.13%  
Moisture Percentage52.39%  
Moisture Percentage57.77%  
Moisture Percentage58.06%  
Moisture Percentage58.46%
```

RESULT:

Thus, soil moisture interfacing with ESP 12E Module nodemcu is calculated and recorded.

INTERFACING NODEMCU WITH RAINDROP SENSOR

Ex.No: 3

Date: 23/1/2026

AIM:

To determine the raindrop value using NodeMCU and raindrop sensor.

PROCEDURE:

- There are three pins in the raindrop sensor namely VCC , A0, GND.
- Connect the VCC pin of sensor to 3.3V of NodeMCU.
- Connect the A0 pin of the sensor to an Analog pin on the NodeMCU.
- Connect the GND pin of sensor to the GND pin of the NodeMCU.

COMPONENTS REQUIRED:

- NodeMCU
- Raindrop Sensor
- Jumper wires

CONNECTION TABLE:

Raindrop Sensor Pin	NodeMCU
Vcc	3.3V
GND	GND
A0	A0

SOURCE CODE:

```
#define POWER_PIN D7 #define AO_PIN    A0
```

```
void setup() { Serial.begin(9600);  
pinMode(POWER_PIN, OUTPUT); }
```

```
void loop() { digitalWrite(POWER_PIN, HIGH);  
               delay(10);  
               int rainValue = analogRead(AO_PIN);
```

```
digitalWrite(POWER_PIN, LOW);  
Serial.println(rainValue); delay(1000); }
```

OUTPUT:

RESULT:

Thus, through the help of raindrop sensor, interfacing with NodeMCU, the rain value is detected and recorded.

INTERFACING ULTRASONIC SENSOR WITH NodeMCU ESP 12E MODULE

Ex.No: 4

Date: 30/1/26

AIM:

To measure the distance of an object using NodeMCU board and ultrasonic sensor.

PROCEDURE:

- There are four pins in the HC-SR04 Ultrasonic sensor.
- The VCC pin powers the sensor while GND is the Ground pin.
- The Trig pin is the Input Pin while Echo is the Output Pin.
- Connect the VCC pin of the sensor to the VIN pin on the NodeMCU.
- Connect the GND pin of the sensor to any GND pin of the NodeMCU.
- Connect the Trig pin of the sensor to D6 (GPIO 12) pin on the NodeMCU.
- Similarly, connect the Echo pin of the sensor to D5 (GPIO 14) pin on the NodeMCU.
- At last, we can check the interfacing of HC-SR04 Ultrasonic sensor with Arduino.

COMPONENTS REQUIRED:

- NodeMCU ESP 12 Module
- HC-SR04 Ultrasonic Sensor
- Bread Board
- Jumper Wires
- Micro USB Cable

CONNECTION TABLE:

NodeMCU	Ultrasonic Sensor
GPIO 12 (D6)	TRIG
GPIO 14 (D5)	ECHO
VIN	VCC
GND	GND

SOURCE CODE:

```
const int trigPin = 12; const int echoPin = 14;
```

```

#define SOUND_VELOCITY 0.034
#define CM_TO_INCH 0.393701

long duration; float distanceCm; float distanceInch;

void setup() { Serial.begin(115200); pinMode(trigPin, OUTPUT); pinMode(echoPin, INPUT);
}

void loop() { digitalWrite(trigPin, LOW); delayMicroseconds(2); digitalWrite(trigPin, HIGH);
delayMicroseconds(10); digitalWrite(trigPin, LOW);

duration = pulseIn(echoPin, HIGH);

distanceCm = duration * SOUND_VELOCITY/2; distanceInch = distanceCm * CM_TO_INCH;

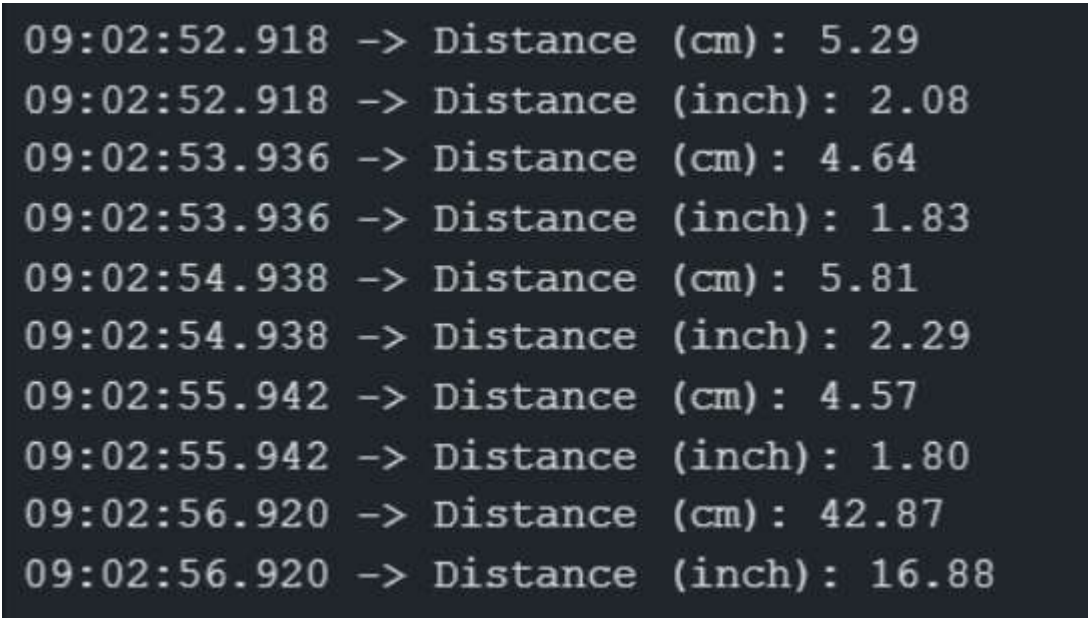
Serial.print("Distance (cm): "); Serial.println(distanceCm);

Serial.print("Distance (inch): "); Serial.println(distanceInch);

delay(1000);
}

```

OUTPUT:



```

09:02:52.918 -> Distance (cm): 5.29
09:02:52.918 -> Distance (inch): 2.08
09:02:53.936 -> Distance (cm): 4.64
09:02:53.936 -> Distance (inch): 1.83
09:02:54.938 -> Distance (cm): 5.81
09:02:54.938 -> Distance (inch): 2.29
09:02:55.942 -> Distance (cm): 4.57
09:02:55.942 -> Distance (inch): 1.80
09:02:56.920 -> Distance (cm): 42.87
09:02:56.920 -> Distance (inch): 16.88

```

RESULT:

Thus, through the help of HC-SR04 Ultrasonic sensor interfacing with NodeMCU, the distance of an object is calculated and recorded in centimetres and inches.

INTERFACING PIR SENSOR WITH nodemcu ESP 12E MODULE

Ex.No: 5

Date: 6/2/2026

AIM:

To detect the motion using nodemcu board and PIR sensor.

PROCEDURE:

- PIR sensors typically have three pins: Vcc, GND, and OUT.
- Connect the Vcc (power) pin of the PIR sensor to the 3.3V pin on the NodeMCU. This provides power to the PIR sensor.
- Connect the GND (ground) pin of the PIR sensor to any GND pin on the NodeMCU.
- Connect the OUT (output) pin of the PIR sensor to any digital pin on the NodeMCU, for example, D1.
- At last, we can check the interfacing of PIR sensor with Arduino

COMPONENTS REQUIRED:

- Nodemcu ESP 12 Module
- PIR Sensor
- Bread Board
- Jumper Wires
- Micro USB Cable

CONNECTION TABLE:

Nodemcu	PIR Sensor
VV, Vin (+3V)	(V) Vcc (positive +)
G, GND (Ground)	(G) GND (Ground -)
Digital Pin(D1)	Vout pin

SOURCE CODE:

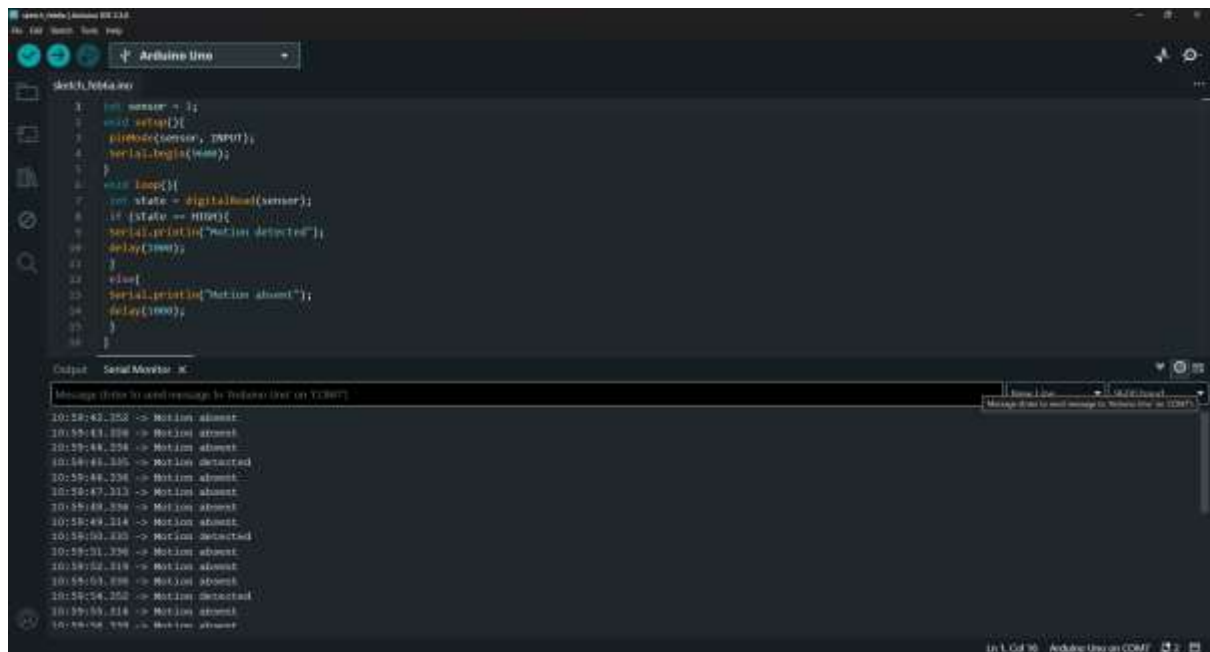
```
int sensor = 4; void setup(){  
pinMode(sensor, INPUT); Serial.begin(9600);  
}  
void loop(){
```

```

int state = digitalRead(sensor); if (state == HIGH){
Serial.println("Motion detected"); delay(1000);
}
else{
Serial.println("Motion absent"); delay(1000);
}
}
}

```

OUTPUT:



The screenshot shows the Arduino IDE interface. The top pane displays the code for a PIR sensor connected to an Arduino Uno. The code defines a sensor pin (12), sets it as an input, and enters a loop that checks the sensor's state. If HIGH, it prints "Motion detected"; if LOW, it prints "Motion absent". The bottom pane shows the serial monitor output, which displays a series of "Motion absent" and "Motion detected" messages with timestamps, indicating the sensor is functioning correctly.

```

1 int sensor = 12;
2 void setup() {
3   pinMode(sensor, INPUT);
4   Serial.begin(9600);
5 }
6
7 void loop() {
8   int state = digitalRead(sensor);
9   if (state == HIGH) {
10    Serial.println("Motion detected");
11    delay(1000);
12  }
13  else {
14    Serial.println("Motion absent");
15    delay(1000);
16  }
17 }

```

Serial Monitor: COM5

```

10:58:42.352 -> Motion absent
10:58:43.398 -> Motion absent
10:58:44.256 -> Motion absent
10:58:45.236 -> Motion detected
10:58:46.236 -> Motion absent
10:58:47.313 -> Motion absent
10:58:48.336 -> Motion absent
10:58:49.336 -> Motion absent
10:58:49.314 -> Motion absent
10:58:50.330 -> Motion detected
10:58:51.336 -> Motion absent
10:58:52.319 -> Motion absent
10:58:53.398 -> Motion absent
10:58:54.252 -> Motion detected
10:58:55.314 -> Motion absent
10:58:56.399 -> Motion absent

```

RESULT:

Thus, through the help of PIR sensor interfacing with nodemcu, the motion is detected and recorded.

INTERFACING ULTRASONIC SENSOR WITH RASPBERRY PI

Ex.No: 6

Date: 13/2/2026

AIM:

To measure the distance using ultrasonic sensor interfaced with Raspberry Pi.

PROCEDURE:

- Connect the VCC (power) pin, GND of the ultrasonic sensor to the 3.3V pin and GND on the Raspberry Pi respectively.
- Connect the TRIG pin of the ultrasonic sensor to GPIO pin 23 on the Raspberry Pi, and connect the ECHO pin of the ultrasonic sensor to GPIO pin 24 on the Raspberry Pi.
- Set the GPIO numbering mode to use BCM numbering.
- Define GPIO pins for TRIG and ECHO.
- Setup TRIG pin as an output and ECHO pin as an input.
- Set TRIG pin to low initially and wait for 2 seconds.
- Send a short pulse to TRIG pin to initiate the ultrasonic signal.
- Measure the time taken for the ultrasonic signal to bounce back.
- Calculate the distance based on the time duration and the speed of sound.
- Print the calculated distance in centimetres.
- Clean up GPIO resources at the end.

COMPONENTS REQUIRED:

- Raspberry Pi (any model with GPIO pins)
- Ultrasonic sensor
- Bread Board
- Jumper Wires

CONNECTION TABLE:

Raspberry Pi	Ultrasonic Sensor
GPIO 23(BCM)	TRIG
GPIO 24 (BCM)	ECHO
3.3V (VCC)	VCC
GND	GND

SOURCE CODE:

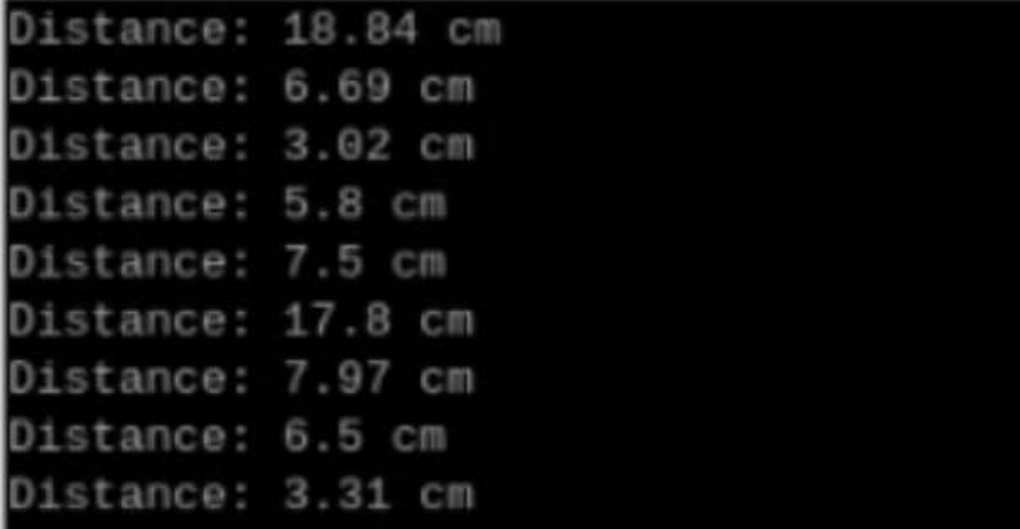
```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)

# Set pin 8 to be an output pin and set initial GPIO.setup(8, GPIO.OUT, initial=GPIO.LOW)
value to low (off)

# Run forever while True:
# Turn on
GPIO.output(8, GPIO.HIGH)
print("LED ON") sleep(1)
GPIO.output(8, GPIO.LOW)
print ("LED OFF") sleep(1) }
```

OUTPUT:



A terminal window with a black background and green text. It displays ten lines of distance measurements in centimeters, each preceded by the label 'Distance:'. The values are: 18.84 cm, 6.69 cm, 3.02 cm, 5.8 cm, 7.5 cm, 17.8 cm, 7.97 cm, 6.5 cm, and 3.31 cm.

Distance (cm)
18.84
6.69
3.02
5.8
7.5
17.8
7.97
6.5
3.31

RESULT:

Thus, the distance to an obstacle detected by the ultrasonic sensor interfaced with Raspberry Pi is accurately measured and printed in centimetres.

INTERFACING SOIL MOISTURE SENSOR WITH RASPBERRY PI 4

Ex.No: 7

Date: 20/2/2026

AIM:

To measure the state of the logic HIGH or LOW based on irrigation system if soil is too dry or moist respectively.

PROCEDURE:

- There are three pins in the soil moisture sensor, one is D0 pin(Digital Output Pin), Vcc power supply pin and one for output data transmission.
- Connect all the pin to Raspberry Pi.
- Connect the D0 pin of the sensor with the GPIO 14 pin of Raspberry Pi.
- Join the GND pin of the sensor to the GND pin of Raspberry Pi 4
- Connect the power supply Vcc pin to the +5 volt pin of Raspberry Pi.

COMPONENTS REQUIRED:

- Raspberry Pi with screen
- Keyboard and Mouse
- Soil Moisture Sensor
- Jumper Wires
- Micro USB Cable
- Red and Green LED

CONNECTION TABLE:

Raspberry Pi 4	Soil Moisture Sensor
GPIO 14 / PIN 8	D0 Pin
GPIO 12 / PIN 32	RGB Red
GPIO 16 / PIN 36	RGB Green
PIN 2 and 4	VCC
PIN 6	GND

```
import RPi.GPIO as gp
gp.Setmode ( gp.BOARD)
gp.setup(8, gPIN)
```

```
gp.setup(36, gp.out, initial=gp.Low)
```

```
gp.setup(32, gp.out, initial=gp.Low)
```

While True:

try:

```
Print(not gp.input(8))
```

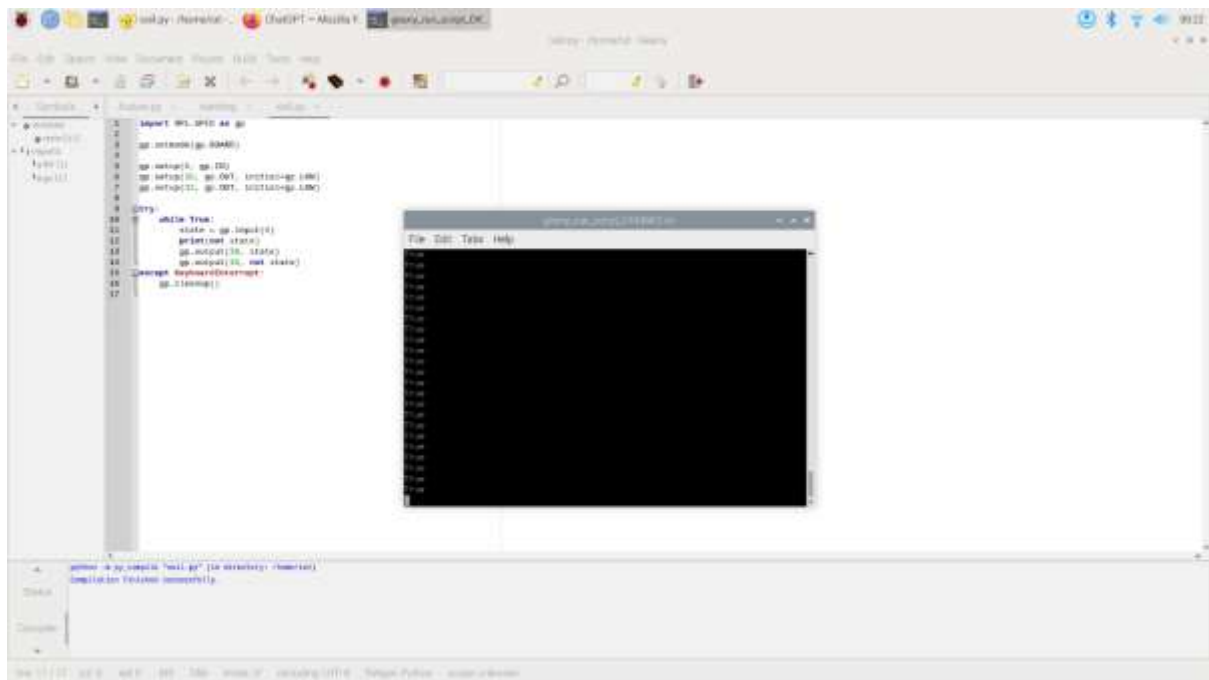
```
gp.output(36, gp.input(8))
```

```
gp.output(32, not gp.input(8))
```

except:

```
gp.cleanup()
```

OUTPUT:



RESULT:

Thus, through the help of soil moisture sensor interfacing with Raspberry Pi 4 , the soil dry or moisture is determined and recorded by HIGH and LOW signal.

INTERFACING OF RASPBERRY PI WITH SINGLE AND MULTI LED BLINK

Ex.No: 8

Date: 27/2/2026

AIM:

To determine the LED blink using Raspberry Pi 4 model.

PROCEDURE:

- LED can be found on more colours and sizes.
- Common LED have two pins. The positive end of a LED is called anode, and negative end called cathode.
- Connect an LED longer LED(anode) of the LED to pin 8 and shorter LED to ground pin (GND) on Raspberry Pi.
- Check interfacing of LED connection with breadboard and Raspberry Pi.
- For multi LED connect one set of LED as above instruction and other set of LED to pin 8 (anode) and shorter LED to ground pin (GND) on Raspberry Pi.

COMPONENTS REQUIRED:

- Raspberry Pi with screen
- GPIO Break out
- Keyboard and Mouse
- Breadboard
- LED (2-multi LED)
- 220 ohm resistor

CONNECTION TABLE:

Raspberry Pi 4	LED
GPIO PIN 8 (BCM 14)	(+) Anode (larger pin)
GPIO PIN 16	(+) Anode (larger pin)
Ground (GND)	(-) Cathode (smaller pin)
Ground (GND)	(-) Cathode (smaller pin)

)

CODE:

For Single - LED blink:

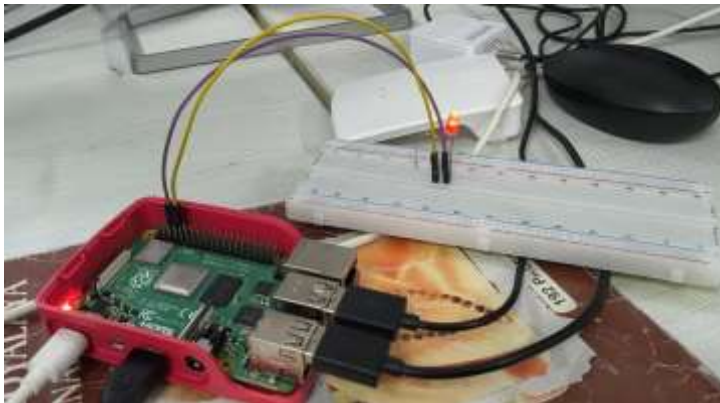
```
import RPi.GPIO as gp from time import sleep gp.setwarnings(False) gp.setmode ( gp.BOARD)
gp.setup(8, gp.out,initial=gp.Low) While True:
gp.output(8,gp.HIGH) Sleep(1) gp.output(8,gp.LOW) Sleep(1)
```

For Multi - LED blink:

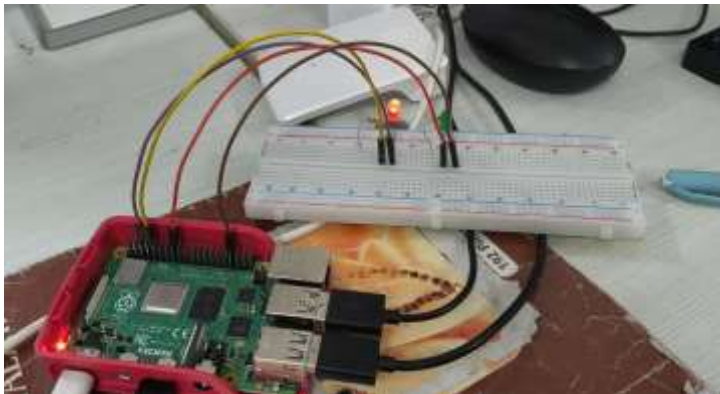
```
import RPi.GPIO as gp from time import sleep gp.setwarnings(False) gp.setmode ( gp.BOARD)
gp.setup(8, gp.out,initial=gp.Low) gp.setup(16, gp.out,initial=gp.Low) While True:
gp.output(8,gp.HIGH) gp.output(16,gp.LOW) Sleep(1) gp.output(8,gp.LOW) gp.output(16,gp.HIGH)
Sleep(1)
```

OUTPUT:

For Single-LED Blink:



For multi-LED Blink:



RESULT:

Thus, through the help of LED interfacing with Raspberry Pi , the LED blink is been viewed as a output for both single and multi LED blink.

INTERFACING RAINDROP SENSOR WITH RASPBERRY PI

Ex.No: 9

Date: 6/3/2026

AIM:

To detect the rainfall using raspberry pi and raindrop sensor.

PROCEDURE:

- Connect the rain sensor module to the Raspberry Pi using jumper wires.
- Connect the VCC pin of the rain sensor module to a 3.3V pin on the Raspberry Pi.
- Connect the GND pin of the rain sensor module to any GND pin on the Raspberry Pi.
- Connect the OUT pin of the rain sensor module to a GPIO pin on the Raspberry Pi.

COMPONENTS REQUIRED:

- Raspberry Pi Module
- Rain Sensor
- Bread Board
- Jumper Wires
- USB Cable

CONNECTION TABLE:

Raspberry Pi	Raindrop Sensor
VV, Vin (+3.3V)	(V) Vcc (positive +)
G, GND (Ground)	(G) GND (Ground -)
GPIO 18	Vout pin

```
from time import sleep

from gpiozero import InputDevice no_rain = InputDevice(18)

while True:

    if not no_rain.is_active:

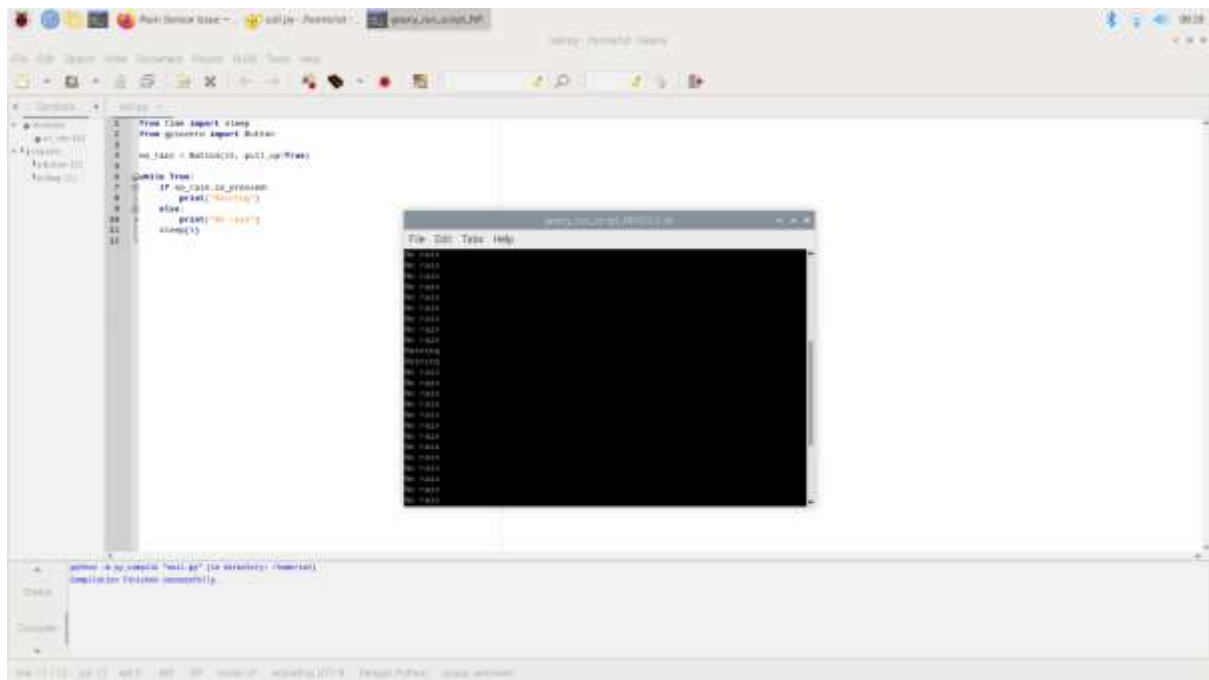
        print("No_rain")

    else:

        print("Raining")

    sleep(1)
```

OUTPUT:



RESULT:

Thus, through the help of raindrop sensor interfacing with Raspberry pi, the rainfall is detected.

INTERFACING RASPBERRY PI WITH BUZZER

Ex.No: 10

Date: 6/3/2026

AIM:

To determine the buzzer sound with no.of times using raspberry pi and buzzer

PROCEDURE:

- There are three pins in the buzzer modules out of which two are for power and one for output data transmission.
- Connect the Vcc (power) pin to a 3.3V (or 5V) pin on Raspberry Pi 4.
- Connect the signal pin to GPIO pin on the Raspberry Pi4, use GPIO pin 23.
- Connect the GND pin to any ground pin (GND) pin on the Raspberry Pi4.
- Check the interfacing of the buzzer modules with Raspberry Pi4.

COMPONENTS REQUIRED:

- GPIO breadboard (optional)
- Breadboard
- Buzzer/Piezo speaker

CONNECTION TABLE:

Buzzer Pin	Raspberry PI4
Vcc	5V (pin 2 or 4)
GND (Ground)	GND (any pin)
Signal	GPIO 18 (BCM 24)

```
import RPI GPIO as GPIO
```

```
from time import sleep
```

```
GPIO.setwarnings(False)
```

```
GPIO.setmode(GPIO.BCM)
```

```
buzzer = 23
```

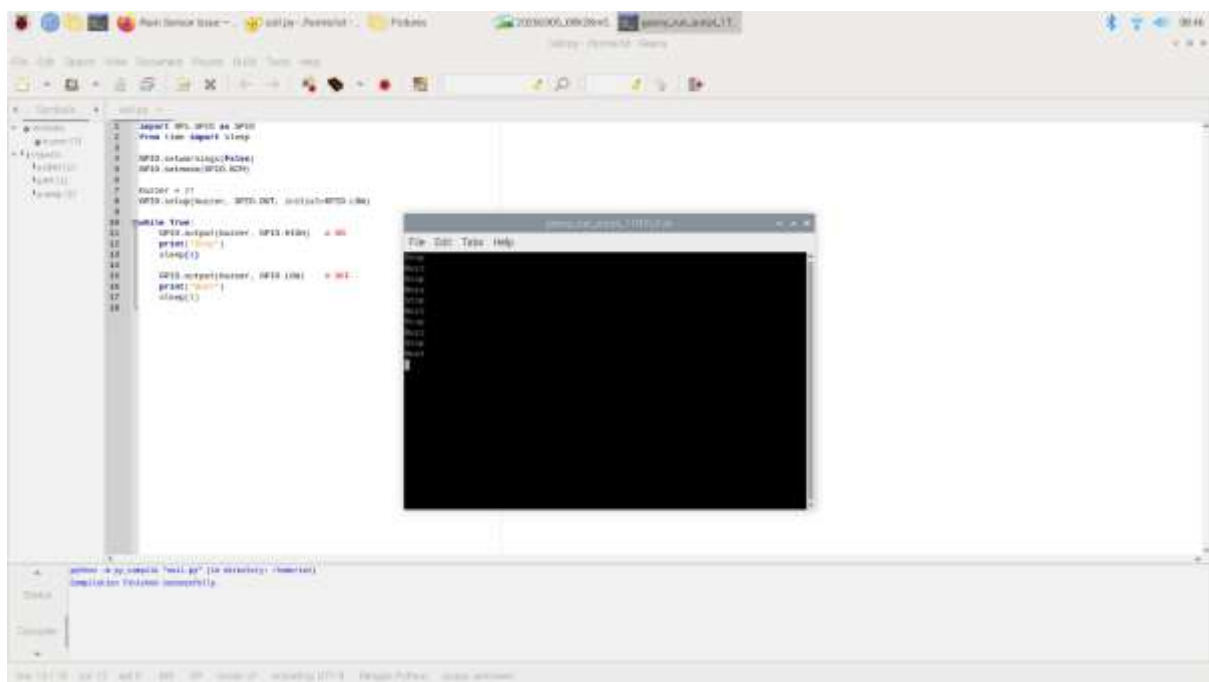
```
GPIO.setup(buzzer,GPIO.OUT)
```

```
while True:
```

```
    GPIO.output(buzzer,GPIO.HIGH)
```

```
print("Beep")  
sleep(0.5)  
GPIO.output(buzzer,GPIO.LOW)  
print("No Beep")  
Sleep(0.5)
```

OUTPUT:



RESULT:

Thus, through the help of buzzer modules, interfacing with raspberry pi4, the buzzer sound is detected and recorded